

# CodeKrafters club

*OWASP Juice shop*

Task 1 - Local Vulnerability Assessment

## CodeKrafters Club – Task 1

Submitted by: Manohari S.J. from CSE B 2nd year

Application: OWASP Juice Shop

Assessment Type: Local Security Assessment

Target: `http://localhost:3000`

Juice Shop Version: 20.2.0

Node.js: v24.21.0

npm: 11.19.0

## 1. DOM XSS

### What I tried

I went to the Juice Shop Score Board and looked through the different vulnerability categories.

I noticed the XSS category and found the challenge called DOM XSS.

The challenge itself gave the following payload:

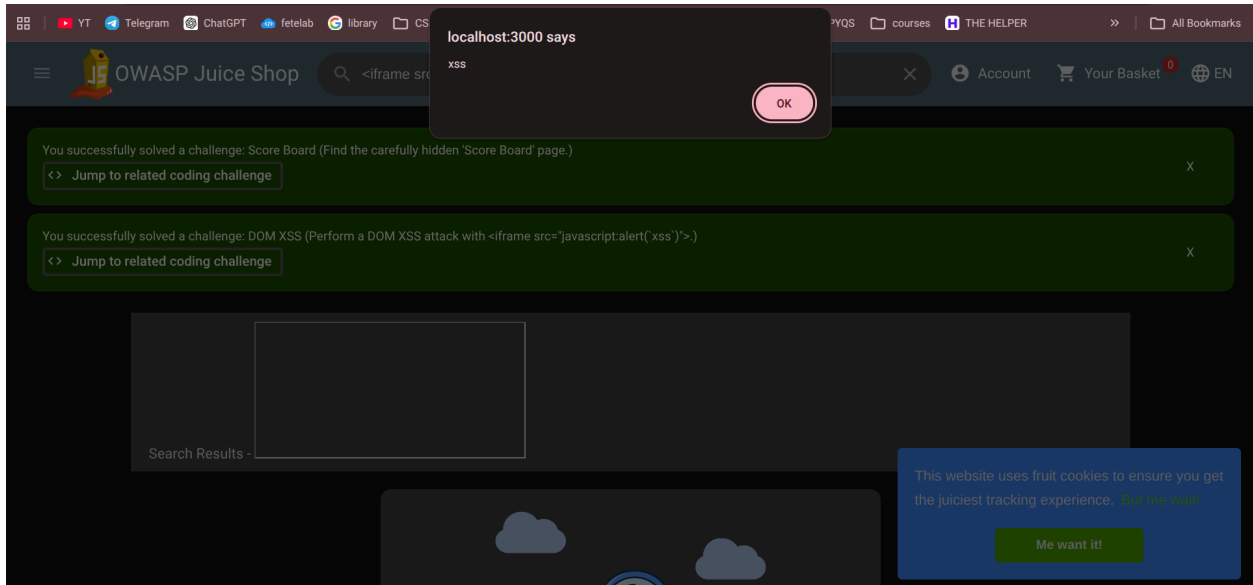
```
<iframe src="javascript:alert(`xss`)">
```

I wanted to understand what would actually happen instead of just copying it, so I tried it through the search box.

A JavaScript popup appeared showing:

xss

This confirmed that the JavaScript inside the injected input had actually executed.



## What I understood

The important thing I understood from this challenge is that the search input was not being treated completely as harmless text.

The browser side code was processing the input in a way that allowed HTML/ JS to execute.

This is called DOM-based XSS.

If this happened in a real website, an attacker could potentially execute JavaScript in another user's browser.

---

## 2. Login Admin – SQL Injection

Next, I went back to the Score Board and checked the Injection category.

I found a challenge called Login Admin.

At first, I understood that the challenge was related to getting access to the administrator account, but I wanted to understand how the login could be bypassed.

I tested the login page using a SQL injection payload.

In the email field I entered:

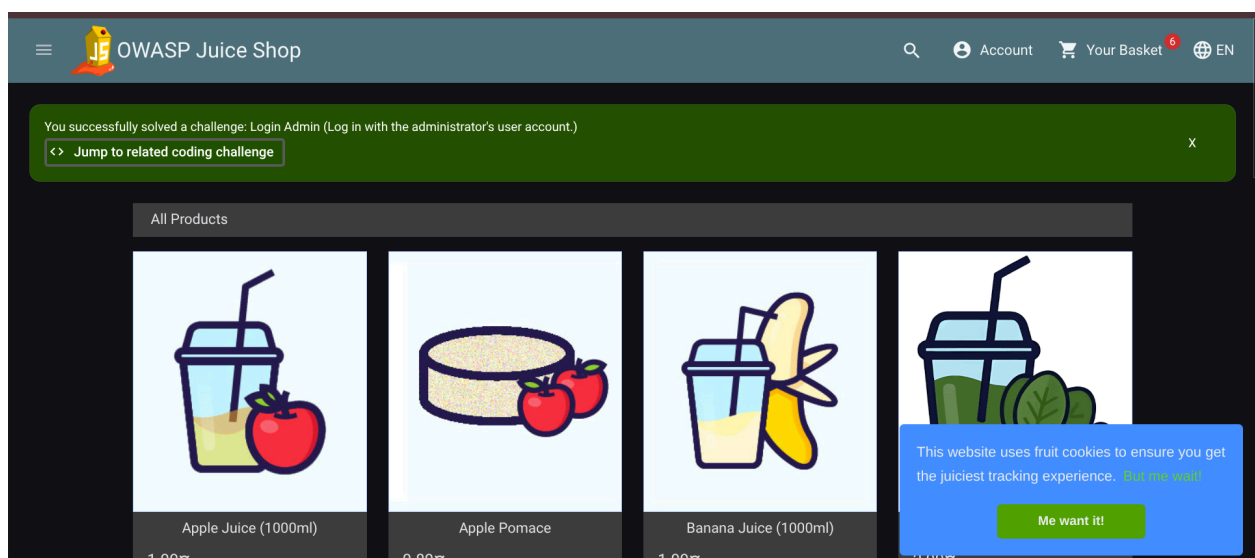
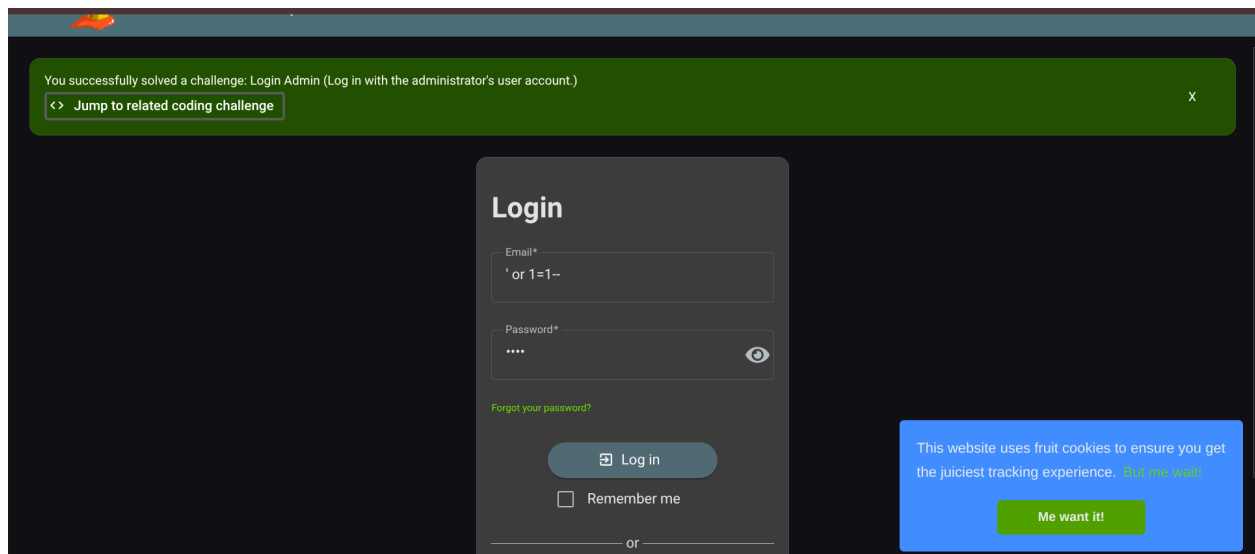
' or 1=1--

For the password, I entered:

test

Then I clicked Login.

The login succeeded.



## Understanding the payload

After seeing it work, I broke the payload down to understand why it worked.

' or 1=1--

The:

,

closes the original SQL string.

The:

OR 1=1

adds a condition that is always true.

And:

--

comments out the remaining part of the SQL statement.

So the original login query can be manipulated into something where the condition becomes true instead of properly checking the password.

## What I understood

This was my first proper hands-on example of SQL Injection.

I understood that the problem is not the payload itself. The real problem is when an application directly places user input into an SQL query without using safe parameterized queries.

---

## 3. Confidential Document

In Sensitive Data Exposure I saw the Confidential Document challenge.

I selected the Confidential Document challenge under the Sensitive Data Exposure category. Since this type of vulnerability can involve sensitive files being exposed through publicly accessible resources, I first looked for file-related paths in the application. The Juice Shop challenge and its available hints pointed toward the application's `/ftp/` directory, so I checked:

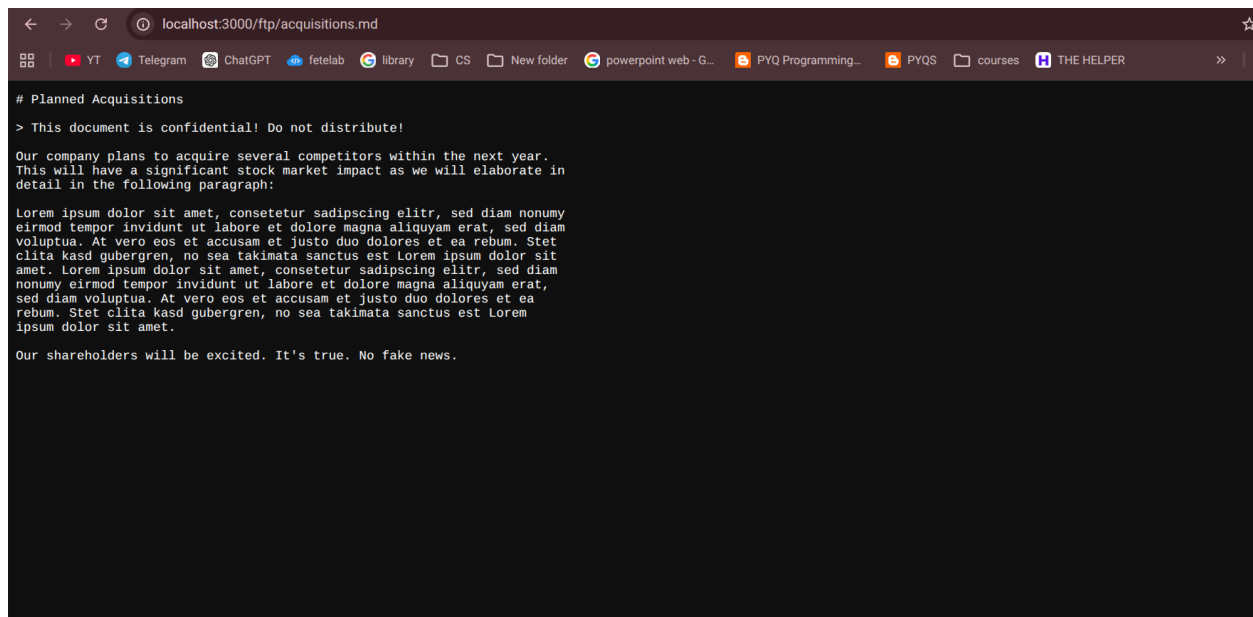
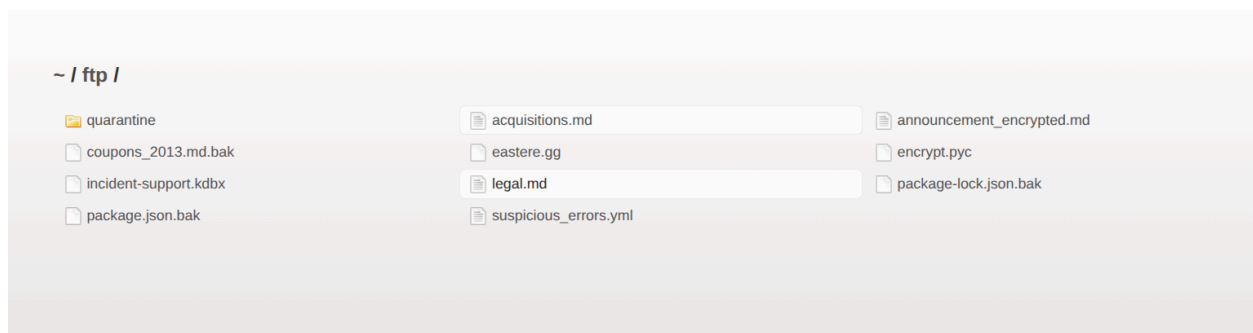
`http://localhost:3000/ftp/`

The `/ftp/` directory was accessible.

While looking through the files, I found:

`acquisitions.md`

I opened it and could access the document.



## What I understood

This showed me that sensitive files can be exposed simply because they are placed somewhere that the web server can access.

Even if the website doesn't provide a visible link to the file, an attacker can sometimes discover directories and files by exploring the application.

In a real application, confidential files should not be placed in publicly accessible locations.

---

## 4. Admin Section – Broken Access Control

The next challenge was Admin Section under Broken Access Control.

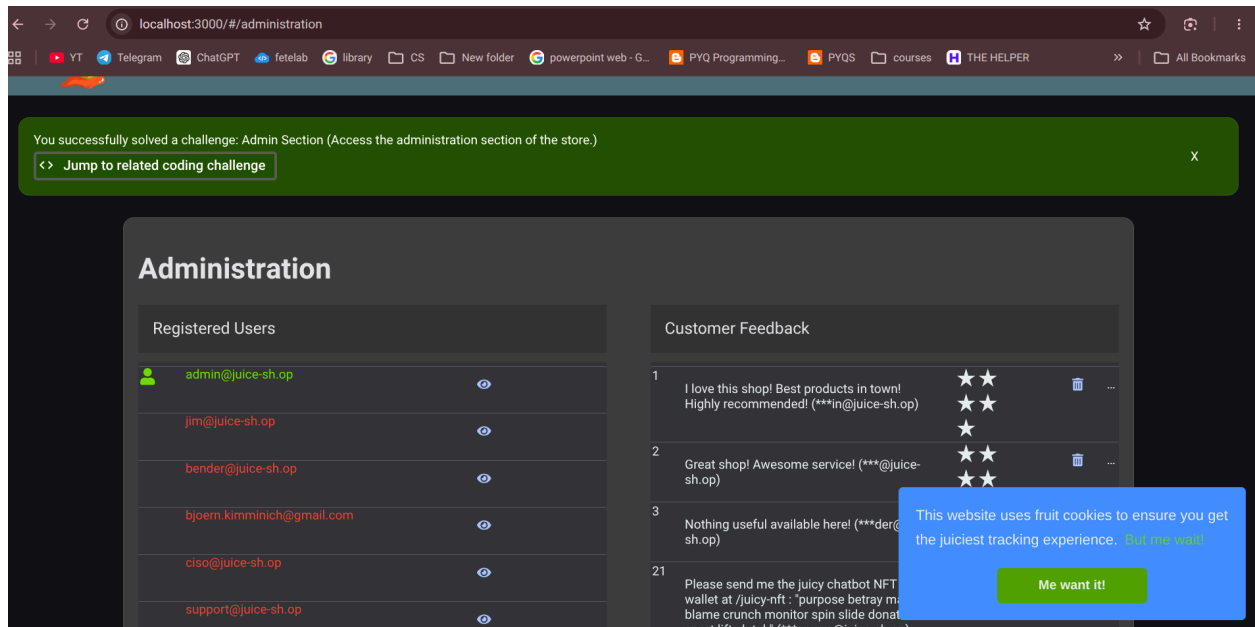
I already knew from the previous challenge that Juice Shop had an administrator area, so I checked whether I could reach the administration page directly.

I tried:

`http://localhost:3000/#/administration`

### What happened

The Administration section opened.



## What I understood

This helped me understand something important about access control.

A website hiding an Admin button does not actually protect the administration functionality.

A user can potentially discover the route and try to access it directly.

Real applications should check the user's permissions when the request is made.

So:

Authentication = Who am I?

Authorization = What am I allowed to access?

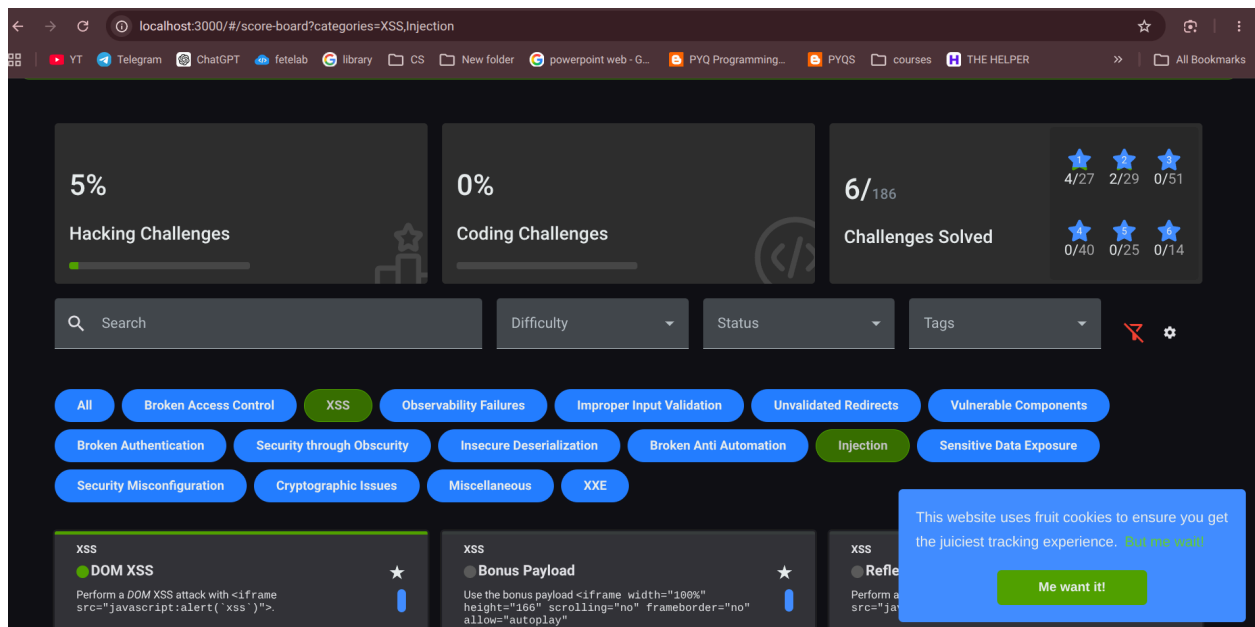
This challenge helped me understand the difference much more clearly.

---

# My Final Progress

After working through the challenges, I checked the Juice Shop Score Board again.

At this point, I had completed challenges across different vulnerability categories.



The four vulnerability areas I worked on for this task were:

- DOM XSS → XSS
- Login Admin → Injection / SQL Injection
- Confidential Document → Sensitive Data Exposure
- Admin Section → Broken Access Control

---

## What I Learned From Doing This Myself

Before doing this task, I mostly knew vulnerabilities like XSS and SQL Injection from definitions and examples.

Actually running the application and trying the challenges made the concepts much clearer.

For example, with DOM XSS, I could actually see the JavaScript alert appear after entering the payload.

With SQL Injection, I understood what each part of:

' or 1=1--

was doing instead of treating it as a random string.

The Confidential Document challenge showed me that sensitive information can be exposed through something as simple as an accessible directory.

The Admin Section challenge helped me understand why hiding a page or button is not the same as implementing proper authorization.

This task also relates to my own **PII and Data Leakage Guardrail project**. Working with Juice Shop helped me understand security from a practical perspective and made me think more about how sensitive data can be exposed when proper security controls are missing. It also gave me some ideas that I can apply to my own project.

Overall, this task gave me my first proper hands-on experience with finding and exploiting web application vulnerabilities in a controlled environment. It also made me understand why secure coding practices such as input validation, parameterized queries, proper authorization, and secure file handling are important.